

ERSkill: 面向技能引导的自适应记忆检索的进化框架

Haolong Chen^{1,2,3}, Liang Zhang⁴, Zhuo Li^{1,2,3}, Lei Xue⁴, Guangxu Zhu^{1,2,3,5}

¹Shenzhen International Center for Industrial and Applied Mathematics

²Shenzhen Research Institute of Big Data

³The Chinese University of Hong Kong, Shenzhen

⁴Shenzhen Campus of Sun Yat-sen University

⁵Shenzhen Loop Area Institute

haolongchen1@link.cuhk.edu.cn, zhangliang27@mail.sysu.edu.cn,
221019088@link.cuhk.edu.cn, xuelel3@mail.sysu.edu.cn, gxzhu@sribd.cn

摘要

尽管大语言模型 (Large Language Model, LLM) 智能体在持续交互中日益依赖长期记忆, 但管理该记忆的检索机制却很少被视为可进化的组件。这种静态方法限制了在异构记忆查询上的性能, 因为此类查询通常需要多样化的证据构建策略。为解决这一问题, 本文提出 **ERSkill**, 一个以检索为核心、支持自我进化且由技能引导的记忆访问框架。**ERSkill** 将交互历史整理为结构化记忆存储, 并将检索行为表示为由基本原语构成的可执行技能。在推理阶段, 训练后的路由器动态地将每个查询匹配到最优技能, 以构建定制化证据用于答案生成。为实现持续改进, **ERSkill** 在训练过程中协同进化技能集与路由器。它采用经验前缀树高效记录已探索的检索路径, 并利用双前沿机制安全地将新技能能力的扩展与面向路由器的稳定部署解耦。在多个智能体记忆基准上的实验表明, **ERSkill** 显著优于非进化和自我进化的强基线方法。值得注意的是, 使用 Qwen3-Next-80B-A3B-Instruct 时, 其在 F1、BLEU-1 和 LLM 评判分数的总体平均值上提升了 31.3%, 使用 GPT-5.4-nano 时提升了 28.1%。

1 引言

大语言模型 (LLM) [1, 2, 3, 4] 智能体日益被期望充当持久协作者, 而非一次性助手 [5]。在跨越数周或数月的长期交互中, 智能体积累用户偏好、跟踪变化事件、记住先前决策并复用经验, 这推动了智能体记忆领域的快速进展 [6]。近期工作通过构建显式外部记忆来存储交互历史, 在实现持久智能体方面取得了进展。这些系统通过信息抽取、压缩、更新和遗忘等操作构建并维护记忆 [7, 8, 9], 并检索已存储的记忆

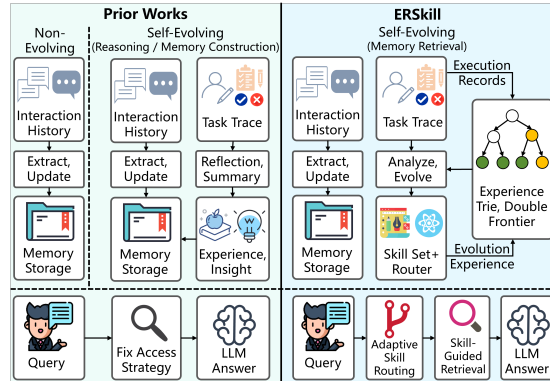


图 1: 左: 先前工作包括用于推理或记忆构建的非进化方法和自我进化方法。右: ERSkill 研究通过技能引导的自适应访问实现自我进化的记忆检索。

作为下游任务的证据。更近期的研究进一步迈向自进化智能体：不再仅存储关键信息，而是反思过去的轨迹，提炼可复用的经验或洞见，存储并应用它们来指导未来行为 [10, 11, 12, 13, 14]。

尽管取得了这些进展，智能体记忆的检索侧作为进化对象仍未得到充分探索。现有的自进化方法主要利用过去的任务经验来改进未来的推理 [10, 11, 12, 13]；例如，推理银行（ReasoningBank）[12] 反思过去的任务迹以提炼可复用的推理经验。记忆技能（MemSkill）[14] 进化基于大语言模型的记忆提取技能，但生成的记忆在查询时仍通过预定义的稠密检索策略访问。因此，虽然记忆内容和推理指导变得越来越自适应，检索行为本身往往仍是预定义的。这对智能体记忆问答构成了限制，因为查询在信息需求上可能存在显著差异。例如，“爱丽丝在夏威夷旅行期间给鲍勃买了什么礼物？”主要需要检索特定事件，而“为什么爱丽丝后来不再计划与鲍勃的另一次夏威夷旅行？”则需要将早期事件与后续发展联系起来以揭示因果关系。此类查询需要性质不同的证据构建行为，从而引出一个关于查询时记忆访问的核心问题：

大语言模型智能体如何进化并学习组合复杂的检索动作，使其检索行为适应不同查询的异构信息需求？

本文提出 **ERSkill**（Evolving Retrieval Skill，进化检索技能），一个以检索为中心的自我进化框架，将智能体记忆访问建模为技能引导的 [15] 证据构建。为支持多样化的检索视角，**ERSkill** 首先构建面向检索的记忆存储，通过检索原语库（如稠密检索 [16]、BM25 检索 [17] 和查询重写 [18]）暴露记忆。基于这些原语，**ERSkill** 将检索行为表示为一组可执行的检索技能。每个技能通过将原语组合成序列来指定具体的检索模式。这种技能抽象使检索行为可复用、可解释且可优化。为每个查询选择合适的检索行为，**ERSkill** 训练一个技能路由器，将查询的信息需求与技能匹配。在推理时，所选技能控制记忆原子如何被收集、扩展和组织成任务特定的证据视图，用于答案生成。通过这种方式，**ERSkill** 实现了查询自适应的记忆访问，同时保持检索过程可执行且可解释。

然而，预定义的静态检索技能集从根本上限制了记忆访问的表达能力，因为检索需求因查询而异。因此，我们设计了一个技能-路由器协同进化过程，通过分析先前的任务迹并识别当前技能行为与期望证据构建之间的差距，联合更新技能集和路由器。为减少低效探索，ERSkill 将探索过的原语路径存储在经验字典树中，使技能生成器能够复用过去的展开经验，避免重复提出等效的检索程序。此外，由于路由器在部署时可能并不总是选择最佳技能，技能进化不仅应考虑技能在原则上是否有用，还应考虑其效用能否被路由器可靠激活。为此，ERSkill 采用帕累托式双前沿机制：能力前沿维护一组具有最佳检索能力的紧凑技能集，而部署前沿维护一个面向路由器的技能集，该技能集在路由推理下经过验证，使 ERSkill 能够在保持部署稳定的同时扩展检索能力。

总之，本文的贡献如下：

- 本文提出一种以检索为核心的智能体记忆框架，将记忆访问视为查询自适应的证据构建，而非固定的检索策略。
- 本文提出 ERSkill，一种技能引导的框架，利用经验字典树和双前沿设计，使检索技能与路由器协同演化。
- 在智能体记忆基准上的实验表明，ERSkill 优于强基线，在 F1、BLEU-1 和大模型作为评审分数上的总体平均值分别提升了 31.3%（使用 Qwen3-Next-80B-A3B-Instruct）和 28.1%（使用 GPT-5.4-nano）。ERSkill 还实现了领先的成本-性能权衡。

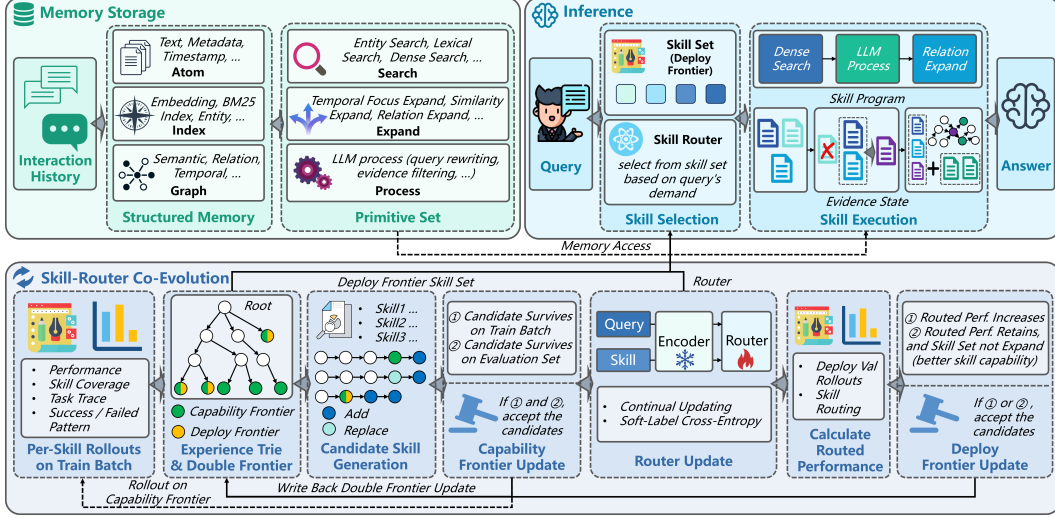


图 2: ERSkill 概览。ERSkill 将交互历史编译为记忆存储, 从原语构建检索技能, 并选择与查询匹配的技能来构建证据以进行回答。在训练过程中, 技能与路由器协同演化, 经验字典树记录已探索路径, 双前沿支持技能能力扩展和面向路由器的部署。

2 方法论

2.1 概览

如图 2 所示, **ERSkill** 通过为每个查询选择与其信息需求匹配的检索技能来适应记忆检索。它首先将交互历史编译为记忆存储, 然后使用训练好的路由器选择最合适的技能, 最后执行所选技能以构建证据进行回答。在训练过程中, 技能与路由器协同演化。

2.2 记忆存储

结构化记忆。 设 D 为交互历史。ERSkill 将 D 拆分为原子级记录 $\mathcal{A} = \{a_1, \dots, a_n\}$, 其中每个 a_i 存储原子文本、元数据和时间戳。编译后的记忆为 $M(D) = (\mathcal{A}, \mathcal{I}, \mathcal{G})$, 其中 $\mathcal{I}$ 是用于原子搜索的索引集合, $\mathcal{G}$ 是用于原子扩展的图集合。索引提供进入候选原子的入口点 (例如, 用于稠密检索的嵌入索引和用于基于实体的检索的实体到原子索引), 而图连接原子以进行扩展 (例如, 基于相似性的扩展)。附录 A.1 总结了构建细节。

检索原语。 ERSkill 的记忆不仅仅是存储, 而是用于构建查询特定证据的可执行基底。具体而言, ERSkill 通过原语库 $\mathcal{P} = \{p_1, \dots, p_m\}$ 暴露记忆存储, 其中每个原语是一个状态转移 $p: (q, s, M(D)) \mapsto s'$ 。这里, q 是查询, s 是当前证据状态, s' 是更新后的状态。

检索原语使用 $\mathcal{I}$ 中的索引从不同角度访问记忆。形式上, 我们初始化: (1) `entity_search` 识别与查询相关的实体并通过实体-原子图检索原子; (2) `lexical_search` 执行 BM25 风格的表面形式匹配; (3) `dense_search` 通过查询-原子嵌入相似性检索原子。此外, 我们定义扩展原语使用 $\mathcal{G}$ 中的图来增长证据状态。具体而言, `temporal_focus_expand` 从给定的时间跨度添加原子, `similarity_expand` 沿相似性边传播, `relation_expand` 沿类型化关系边传播。我们还包括 `llm_process` 用于查询重写、证据过滤和其他动态控制操作。原语库在整个演化过程中保持不变; 技能仅在如何组合这些原语方面有所不同。由于所有技能共享相同的原语, 它们的结果可以累积在共同的经验结构中。附录 A.1 提供了原语库的详细信息。

Skill: entity-anchored-similarity-expansion

Description:

Designed for entity-specific questions that require evidence beyond direct entity mentions, such as related context, consequences, or follow-up developments. This skill anchors retrieval on the mentioned entities, selects relevant retrieved atoms as expansion seeds, and expands to semantically similar atoms for additional support.

Information Preference:

Prefer entity-grounded evidence related to the queried event or decision. The expansion seeds should be atoms whose similar neighbors may reveal follow-up context, outcomes, or later developments.

Program:

1. **Primitive:** entity_search. Query: {question}.
 2. **Primitive:** llm_process. Process Prompt: "Given the question and the current entity-grounded evidence, select the atom IDs that should be expanded to find semantically related supporting context. Prefer atoms that directly mention the target entities and are closest to the queried event or decision." Output: seed_atom_ids.
 3. **Primitive:** similarity_expand. Atom IDs: {seed_atom_ids}.

图 3: 技能示例, 展示了以实体为锚点的相似性扩展路径。该技能首先检索以实体为基础的原子, 然后选择种子原子, 最后扩展到语义相似的原子以提供支持上下文。

2.3 推理

检索技能。检索技能是一种可执行的原语程序 $\kappa = (c_\kappa, \rho_\kappa)$, 其中 c_κ 是技能描述与信息偏好, $\rho_\kappa = (p_{\kappa,1}, \dots, p_{\kappa,L_\kappa})$ 是原语序列。给定查询 q , 该技能通过应用 $s_j = p_{\kappa,j}(q, s_{j-1}, M(D))$ 顺序执行其原语, 并返回 s_{L_κ} 作为证据视图。状态维护当前证据集与执行上下文, 使技能能够根据查询的证据需求组织检索到的信息。每个技能存储为一个标记文件; 图 3 展示了一个示例。

技能路由器。ERSkill 采用查询条件路由模型来选择检索技能。给定查询 q 和检索技能集 $\mathcal{K}$, 路由器根据每个技能与查询的兼容性为其分配相关性分数。具体而言, 我们使用共享编码器 $\text{Enc}(\cdot)$ 将查询 q 和每个技能 κ 编码为向量表示, 得到 $h_q = \text{Enc}(q)$ 和 $h_\kappa = \text{Enc}(\kappa)$, 其中 $\kappa \in \mathcal{K}$ 。然后使用可学习的评分函数 $u_\theta(q, \kappa)$ 来衡量查询 q 与技能 κ 之间的兼容性, 反映将技能 κ 应用于查询 q 的性能。路由模型 $R_\theta(\cdot)$ 通过将这些分数归一化为技能上的概率分布来定义:

$$R_\theta(\kappa | q, \mathcal{K}) = \frac{\exp(u_\theta(q, \kappa))}{\sum_{\kappa' \in \mathcal{K}} \exp(u_\theta(q, \kappa'))}, \quad (1)$$

这使得路由器能够根据新演化技能的文本描述、信息偏好和程序对其进行评分, 而无需改变输出空间。我们保持文本编码器冻结, 仅优化路由参数。

技能路由与答案生成。给定查询 q 和技能集 $\mathcal{K}$, 路由器选择最匹配查询信息需求的技能。我们将该路由技能表示为 $\pi_\theta(q; \mathcal{K}) = \arg \max_{\kappa \in \mathcal{K}} R_\theta(\kappa | q, \mathcal{K})$, 并在推理时写作 $\hat{\kappa} = \pi_\theta(q; \mathcal{K})$ 。然后 ERSkill 在 $M(D)$ 上执行 $\hat{\kappa}$ 以细化证据状态 $s_{L_{\hat{\kappa}}}$, 并生成答案 $\hat{y} = f_{\text{LLM}}(q, s_{L_{\hat{\kappa}}})$ 。通过这种方式, ERSkill 通过技能选择与执行实现了自适应的查询时记忆访问。

2.4 技能-路由器协同演化

ERSkill 通过两条帕累托式前沿 (Pareto-style Frontiers) 演化技能。前沿是已探索技能的活跃边界, 保留演化历史中紧凑且非冗余的技能。能力前沿 $\mathcal{C}_t$ 保留具有预言机侧价值的技能, 即当每个查询都能在没有路由器错误的情况下选择最佳技能时, 能够达到最佳可获取效用的技能。因此, 它追踪迄今为止发现的检索能力。部署前沿 $\mathcal{B}_t$ 是推理时面向路由器的技能集, 仅包含经过验证门控的、路由器能够可靠选择的技能。这种分离使 ERSkill 能够探索新的检索能力, 同时仅暴露稳定的技能用于部署。

在演化步骤 t , ERSkill 拥有经验字典树 $\mathcal{T}_t$ 、前沿 $\mathcal{C}_t$ 和 $\mathcal{B}_t$, 以及路由器参数 θ_t 。演化从三个单原语种子技能开始: 语义线索 (semantic-clue) dense_search、实体聚焦 (entity-focus) entity_search, 和表面事实 (surface-fact) lexical_search; 两个前沿均由它们初始化, 因此 $\mathcal{B}_0 = \mathcal{C}_0$ 。给定一个训练批次 $\mathcal{Q}_t$, ERSkill 在 $\mathcal{C}_t$ 中的每个技能上对每个查询执行, 并记录查询-技能性能、执行轨迹和能力重叠于

$\mathcal{T}_t$ 。这些经验揭示了当前能力前沿的薄弱或冗余区域，指导技能候选生成、能力前沿更新、路由器更新和部署前沿更新。

附录 B.1 总结了完整的协同演化算法。

技能候选生成。 ERSkill 通过编辑当前能力前沿的路径来生成新的技能候选，例如添加或替换原语。为了重用过去的搜索经验并避免重复探索，ERSkill 在原始路径上维护一个经验字典树 $\mathcal{T}$ 。由于技能是固定原语库 $\mathcal{P}$ 上的序列，每个根到节点的路径代表一个原语前缀，共享前缀仅存储一次。¹ 这使 ERSkill 能够检测重复候选。

$\mathcal{T}$ 中的每条已探索路径对应一个技能，并存储其训练批次回放、验证回放和前沿状态。技能生成器根据技能摘要、回放统计、失败/成功模式以及字典树摘要提出候选，同时排除 $\mathcal{T}_t$ 中已记录的路径。因此，被接受和被拒绝的候选都保留下来，以指导后续演化。附录 B.2 详细说明了生成过程，附录 E.1 展示了一个示例字典树。

能力前沿更新。 ERSkill 通过保留具有独特预言机侧价值的技能来更新能力前沿。对于每个查询-技能对，我们使用分数 $r(q, \kappa) \in [0, 1]$ 来衡量性能，在实验中实例化为大模型作为评审 (LLM-as-a-Judge) 的准确率；评审提示词见附录 D.4.2。对于一个批次 $\mathcal{Q}$ ，技能效用为 $\text{Util}(\kappa; \mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} r(q, \kappa)$ ，技能集 $\mathcal{K}$ 的预言机分数概况为 $g_{\mathcal{K}}(q) = \max_{\kappa \in \mathcal{K}} r(q, \kappa)$ 。ERSkill 使用前沿重计算算子 $\Phi(\mathcal{K}; \mathcal{Q})$ ，该算子按效用对技能排序，并且仅当移除某个技能不会降低任何查询的 $g_{\mathcal{K}}(q)$ 时才移除它。因此， Φ 在剪枝冗余技能的同时保持了预言机性能。

给定生成的候选技能 $\mathcal{U}_t$ ，ERSkill 首先计算训练侧临时前沿 $\mathcal{C}_{t+1} = \Phi(\mathcal{C}_t \cup \mathcal{U}_t; \mathcal{Q}_t)$ ，并拒绝未保留在其中的候选技能。保留的候选技能 $\mathcal{V}_t$ 随后在验证数据上进行评估，能力前沿更新为 $\mathcal{C}_{t+1} = \Phi(\mathcal{C}_t \cup \mathcal{V}_t; \mathcal{Q}_{\text{val}})$ 。这种两阶段更新利用训练批次进行高效过滤，并利用验证数据进行稳定的前沿重计算。

路由器更新。 训练使用一个窗口 $\mathcal{W}_t$ ，其中包含展开实例 $(q, \mathcal{K}_q, \{r(q, \kappa)\}_{\kappa \in \mathcal{K}_q})$ ， $\mathcal{K}_q$ 是用于该次展开的历史技能集。设 $\mathcal{M}$ 为从 $\mathcal{W}_t$ 中采样的小批量实例。对于每个实例，展开分数在其关联技能集 $\mathcal{K}_q$ 上诱导出一个软目标分布，路由器使用软标签交叉熵进行优化：

$$\tilde{p}(\kappa | q, \mathcal{K}_q) = \frac{\exp(r(q, \kappa))}{\sum_{\kappa' \in \mathcal{K}_q} \exp(r(q, \kappa'))}, \mathcal{L}_{\text{router}} = - \sum_{(q, \mathcal{K}_q, \cdot) \in \mathcal{M}} \sum_{\kappa \in \mathcal{K}_q} \tilde{p}(\kappa | q, \mathcal{K}_q) \log R_{\theta}(\kappa | q, \mathcal{K}_q).$$

部署前沿更新。 在更新路由器之后，ERSkill 仅在能力前沿发生变化时刷新部署前沿，使用新保留的候选技能 $\mathcal{H}_t = \mathcal{U}_t \cap \mathcal{C}_{t+1}$ ；否则，设置 $\mathcal{B}_{t+1} = \mathcal{B}_t$ 。对于任何技能集 $\mathcal{K}$ ，给定所有 $q \in \mathcal{Q}$ 和 $\kappa \in \mathcal{K}$ 的可用展开分数，其路由性能为 $\text{Routed}(\mathcal{K}, \theta; \mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} r(q, \pi_{\theta}(q; \mathcal{K}))$ 。ERSkill 构造一个候选部署前沿 $\mathcal{B}'_t = \Phi(\mathcal{B}_t \cup \mathcal{H}_t; \mathcal{Q}_{\text{val}})$ ，并计算 $\Delta_{\text{route}}(\mathcal{B}'_t; \theta_{t+1}) = \text{Routed}(\mathcal{B}'_t, \theta_{t+1}; \mathcal{Q}_{\text{val}}) - \text{Routed}(\mathcal{B}_t, \theta_{t+1}; \mathcal{Q}_{\text{val}})$ 。如果 $\Delta_{\text{route}}(\mathcal{B}'_t; \theta_{t+1}) \geq \gamma_{\text{route}}$ ，或者如果 $\Delta_{\text{route}}(\mathcal{B}'_t; \theta_{t+1}) \geq -\xi_{\text{drop}}$ 且 $|\mathcal{B}'_t| \leq |\mathcal{B}_t|$ ，则接受 $\mathcal{B}'_t$ 并设置 $\mathcal{B}_{t+1} = \mathcal{B}'_t$ ，其中 γ_{route} 是路由增益裕度， ξ_{drop} 是紧凑性容差，允许小幅路由性能下降以接受更强、更紧凑的部署技能集。如果候选被拒绝，ERSkill 设置 $\mathcal{B}_{t+1} = \mathcal{B}_t$ 。接受/拒绝结果被写回经验字典树，使后续技能生成能够考虑路由器感知的部署性能。训练结束后，最终的部署前沿用于推理。

命题 2.1 (预言机安全的两级前沿更新)。将预言机覆盖率表示为 $\text{OCov}(\mathcal{K}; \mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} g_{\mathcal{K}}(q)$ 。对于每个演化步骤 t ，ERSkill 满足 $\text{OCov}(\mathcal{C}_{t+1}; \mathcal{Q}_{\text{val}}) \geq \text{OCov}(\mathcal{C}_t; \mathcal{Q}_{\text{val}})$ 且 $\text{OCov}(\mathcal{B}_{t+1}; \mathcal{Q}_{\text{val}}) \geq \text{OCov}(\mathcal{B}_t; \mathcal{Q}_{\text{val}})$ 。

命题 2.1 表明，两个前沿在验证集上均保持非递减的预言机覆盖率，尽管部署前沿可能因路由器感知验证而滞后于能力更新。附录 C 提供了证明。

¹ 尽管 llm_process 可能以不同的提示实例化，我们将其视为一种原始类型，因为 ERSkill 侧重于以检索为中心的证据构建，而非以提示为中心的控制。

Methods	LoCoMo			LongMemEval*			PerLTQA			Average		
	F1	B1	L-J	F1	B1	L-J	F1	B1	L-J	F1	B1	L-J
Qwen3-Next-80B-A3B-Instruct												
<i>Non-Evolving</i>												
A-Mem	32.96	37.20	43.63	15.13	11.10	48.73	42.37	37.94	42.65	30.15	28.75	45.00
MemoryOS	34.03	37.43	46.49	18.15	12.60	49.74	<u>43.70</u>	36.45	46.79	31.96	28.83	47.67
LightMem	42.63	42.89	<u>64.96</u>	16.48	11.00	<u>55.83</u>	43.14	35.30	39.54	34.08	29.73	<u>53.44</u>
<i>Self-Evolving</i>												
Dynamic Cheatsheet	34.96	33.22	56.05	<u>33.59</u>	<u>34.13</u>	51.26	35.50	<u>40.88</u>	45.96	<u>34.68</u>	<u>36.08</u>	51.09
ReasoningBank	15.77	9.12	55.41	17.02	12.85	54.31	43.05	37.55	<u>48.44</u>	25.28	19.84	52.72
GEPA	10.35	4.58	57.00	11.55	7.15	52.28	42.64	37.36	47.41	21.51	16.36	52.23
MemSkill	43.25	<u>44.01</u>	60.19	11.35	6.93	49.74	40.97	32.93	34.78	31.86	27.96	48.24
ERSkill	49.96	50.18	70.06	46.79	53.31	59.39	51.89	44.07	54.45	49.55	49.19	61.30
GPT-5.4-nano												
<i>Non-Evolving</i>												
A-Mem	30.40	29.33	49.04	17.07	13.77	42.13	36.06	32.95	42.65	27.84	25.35	44.61
MemoryOS	33.46	32.72	<u>59.55</u>	15.50	10.57	45.17	43.88	<u>37.27</u>	47.20	<u>30.95</u>	26.85	50.64
LightMem	<u>36.43</u>	37.63	58.28	15.95	10.73	<u>50.25</u>	39.91	33.26	37.88	30.76	<u>27.21</u>	48.80
<i>Self-Evolving</i>												
Dynamic Cheatsheet	18.33	12.57	<u>59.55</u>	21.19	18.31	43.14	43.44	36.46	50.72	27.65	22.45	51.14
ReasoningBank	17.65	11.88	57.96	18.88	13.34	49.74	43.14	36.16	48.44	26.56	20.46	52.05
GEPA	18.80	12.31	<u>59.55</u>	<u>21.74</u>	<u>18.58</u>	<u>50.25</u>	40.41	36.15	<u>51.34</u>	26.98	22.35	<u>53.71</u>
MemSkill	25.39	24.80	58.59	18.17	13.05	46.19	42.23	35.57	47.82	28.60	24.47	50.87
ERSkill	38.68	<u>33.46</u>	72.93	38.89	41.01	56.35	45.35	38.40	54.04	40.97	37.62	61.11

表 1: 在 LoCoMo、LongMemEval 和 PerLTQA 上的主要比较结果。我们报告 F1、BLEU-1 (B1) 和大模型作为评审 (L-J) 分数 (以百分比表示)；数值越高表示性能越好。* 表示 ERSkill 从 LoCoMo 迁移而来, 未在 LongMemEval 上训练。

3 实验

3.1 实验设置

基准。我们在三个智能体记忆基准上评估 ERSkill：LoCoMo[19]、LongMemEval[20] 和 PerLTQA[21]。LoCoMo 和 LongMemEval 包含多会话对话历史，而 PerLTQA 进一步评估在对话历史之外的异构来源上的记忆使用。数据集的详细描述见附录 D.1。

基线。我们将 ERSkill 与强基线进行比较，包括：（1）非演化基线包括 A-Mem [7]、MemoryOS [8] 和 LightMem [9]，它们专注于记忆构建和维护，而不从过去任务迹中进行自我演化；（2）自演化基线包括 Dynamic Cheatsheet [11]、ReasoningBank [12]、GEPA [13] 和 MemSkill [14]。Dynamic Cheatsheet、ReasoningBank 和 GEPA 反思过去任务迹，以提炼可复用的经验或洞见用于未来任务，而 MemSkill 演化基于 LLM 的记忆构建技能用于下游问答。Dynamic Cheatsheet、ReasoningBank 和 GEPA 使用标准 RAG [22, 16] 记忆存储。

实现 细节。我们在两个大语言模型骨干上评估性能：Qwen3-Next-80B-A3B-Instruct [4] 和 GPT-5.4-nano [23]，使用 GPT-4o-mini 作为评判模型。我们报告 F1、BLEU-1 (B1) 和大语言模型评判分数 (L-J，提示见附录 D.4.2)，其中数值越高表示与真实值的对齐越好。对于 ERSkill，我们将演化的训练批大小分别设为 LoCoMo 的 20 和 PerLTQA 的 40。查询和技能使用 Qwen3-Embedding-0.6B [24] 作为 Enc(·) 进行编码。对于路由器，两个嵌入首先通过线性层投影到表示空间中。得到的表示为



图 5: 在 LoCoMo 上与 GPT-5.4-nano 的成本-性能比较。左图: L-J 分数与每个历史样本的记忆构建令牌数。省略了不使用基于 LLM 的记忆构建的方法。右图: L-J 分数与每个问答的推理令牌数。ERSkill 在记忆构建和推理两方面都保持轻量级的同时, 实现了最佳性能。

然后拼接并输入到两层多层感知机 (MLP) 中, 为每个查询-技能对生成标量得分 $u_\theta(q, k)$ 。ERSkill 训练一个训练轮次。对于稠密记忆检索, 我们使用 Contriever[25] 作为所有方法的嵌入器。更多细节见附录 D。

3.2 比较实验

ERSkill 达到了最强的整体性能。

表 1 报告了主要比较结果, 其中我们观察到 ERSkill 在两种骨干模型下均取得了最佳总体平均值。具体而言, 与最强基线相比, ERSkill 在使用 qwen3-next-80b-a3b-instruct 时, 将 F1、BLEU-1 和 L-J 的总体平均值提高了 31.3%, 在使用 gpt-5.4-nano 时提高了 28.1%。与非进化方法相比, ERSkill 表明其记忆存储具有足够的信息量, 而技能引导的自适应检索能够提取合适的证据。对于自进化方法, 增益表明当查询时访问保持固定时, 仅更新摘要、提示或推理轨迹是不够的; ERSkill 通过经验树和技能路由器协同进化机制利用检索路径经验获得收益。

如图 4 所示, ERSkill 的优势在需要准确定位特定证据点的任务中更为显著, 例如单跳和多跳, 这表明其对检索行为的有效调节带来了证据搜索能力的提升。

ERSkill 能够跨数据集有效迁移。 ERSkill 还表现出强大的可迁移性。在 LongMemEval 上, 我们直接复用 LoCoMo 上训练的路由器和检索技能, 无需额外训练。如表 1 所示, 在这种迁移设置下, ERSkill 在两种大语言模型骨干下仍取得了最佳性能, 这表明学习到的检索技能捕获了可复用的检索行为。

ERSkill 实现了领先的成本-性能权衡。 图 5 比较了记忆构建和推理的令牌成本。Dynamic Cheatsheet、ReasoningBank 和 GEPA 被从构建成本图中省略, 因为它们使用标准的块嵌入记忆存储而非基于大语言模型的构建。ERSkill 实现了有利的成本-性能权衡: 它是基于大语言模型的记忆构建方法中最轻量的, 因为它仅将大语言模型用于记忆间的关系抽取

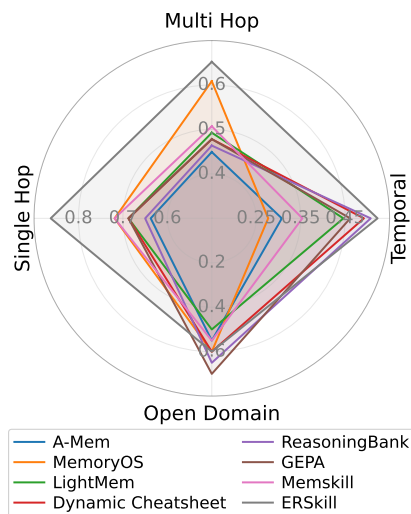


图 4: LoCoMo 类别上的细粒度比较 (GPT-5.4-nano, L-J)。ERSkill 在证据密集型任务 (如单跳和多跳) 上表现出更大的增益。

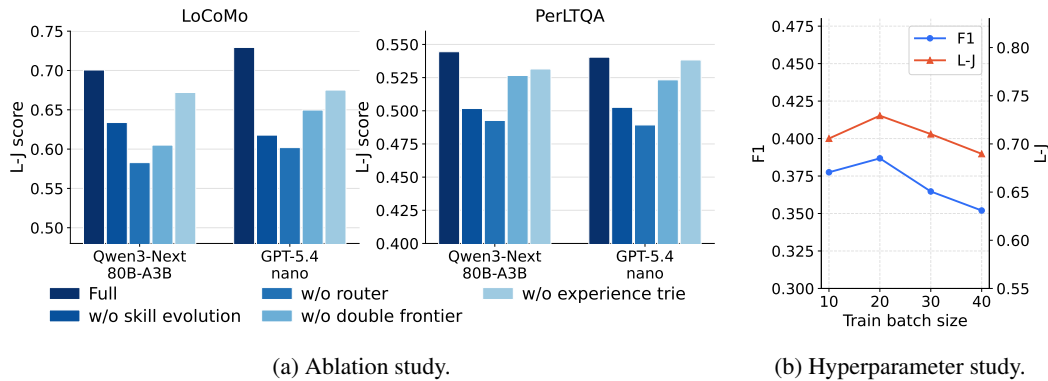


图 6: 消融和超参数研究。左图: 使用两种骨干大语言模型在 LoCoMo 和 PerLTQA 上的消融研究。右图: 使用 GPT-5.4-nano 在 LoCoMo 上对训练批大小的超参数研究。

原子，并在推理时保持在较低成本层级，同时取得最高的 L-J 分数。这表明 ERSkill 的收益来自有针对性的证据构建，而非简单地输入更多检索内容。

3.3 消融实验

消融实验结果验证了每个设计组件的有效性。我们消融了四个核心组件。无技能演化仅保留初始种子技能。无路由器用基于大语言模型的技能选择替代学习到的路由器，其提示词见附录 D.4.3。无双重前沿移除能力前沿与部署前沿，并接受所有生成的技能候选。无经验字典树仅从当前前沿生成候选，不使用历史路径级记录。如图 6a 所示，所有消融变体在数据集和骨干模型上均不如完整 ERSkill。移除技能演化和路由器带来的性能下降最大，表明发现有效检索技能和学习查询相关技能选择的重要性。双重前沿和经验字典树也有贡献：前者稳定技能接受，后者减少重复探索，并利用累积的路径级经验指导候选生成。

3.4 超参数研究

本节研究训练批大小，它控制技能演化的粒度。ERSkill 对训练集进行一次遍历。较大的训练批为每个演化步骤提供更多 rollout 证据，使前沿重计算和路由器更新更稳定，但也减少了演化步骤数。相反，较小的批允许更频繁的技能更新，但每次更新基于不太可靠的 rollout 统计。如图 6b 所示，批大小为 20 在每步稳定演化和足够更新频率之间取得良好平衡。

3.5 演化案例研究

ERSkill 在保持演化可控的同时扩展检索能力。图 7 展示了在 LoCoMo 上使用 GPT-5.4-nano 的一次代表性运行。左图显示，能力前沿和部署前沿的预言机准确率均稳步上升，表明前沿重计算保留并提升了预言机侧的技能覆盖能力。路由部署准确率可能在部署前沿用更高覆盖率的技能替换较弱或冗余技能后暂时下降，此时路由器尚未完全适应。随后的更新会恢复路由性能。这表明 ERSkill 能够在不造成持续部署不稳定的情况下扩展检索能力，同时避免部署前沿过于保守。右图显示，在整个演化过程中前沿规模保持可控。能力前沿仅在新增技能带来非冗余的预言机价值时增长，而部署前沿更为保守。ERSkill 避免累积所有生成的技能，一旦较弱或冗余技能的效用被更强的替代方案覆盖，便将其剪枝。

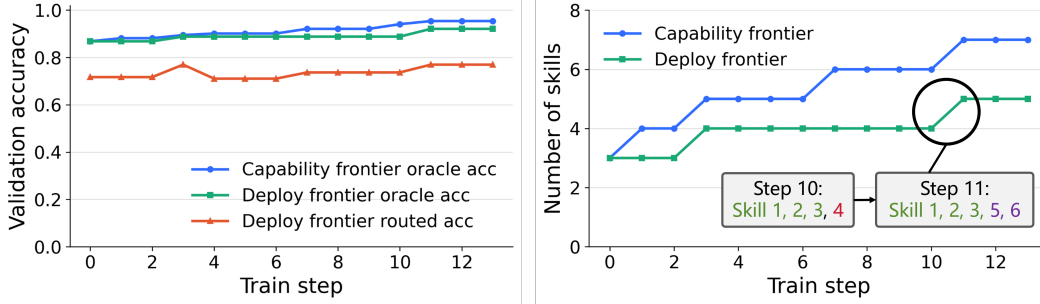


图 7: 演化案例研究。左图: 能力前沿和部署前沿的预言机准确率稳步上升。路由部署准确率可能暂时下降但随后恢复, 表明技能与路由器协同演化稳定。右图: 前沿规模保持可控, 较强技能替换较弱或冗余技能。

3.6 演化稳定性分析

ERSkill 在不同运行间演化稳定。本节评估 ERSkill 技能演化在独立运行中的稳定性。表 2 总结了在 LoCoMo 上使用 GPT-5.4-nano 的五次运行, 并与最强基线进行比较。所有度量的变异系数 (CV) 均较低, 最大值低于 5.4%。这表明 ERSkill 演化具有鲁棒性。

	F1	B1	L-J
均值	38.20	33.69	70.38
标准差	1.36	1.82	1.98
变异系数	3.56%	5.39%	2.81%

表 2: 稳定性分析。

4 相关工作

4.1 智能体记忆

记忆系统使智能体能够检索历史信息, 并将其作为后续推理与决策任务的证据。近期关于智能体记忆的研究探讨了如何将长交互历史外部化为持久记忆存储。典型流程将交互分解为记忆原子, 压缩或整合显著信息, 将所得记忆存储于外部存储中, 并在未来查询到来时检索相关证据 [26, 27, 7, 9, 8]。部分方法专注于设计更有效的记忆流程, 如 A-MEM [7] 与 LightMem [9]; 另一些方法则通过训练提升智能体的记忆管理能力, 如 Memory-R1 [28]、Mem- α [29] 与 MemAgent [30]。尽管取得了这些进展, 大多数方法在查询时仍通过预定义的检索策略暴露记忆。相比之下, 本文聚焦于自适应记忆检索: 智能体应根据异构查询需求访问、扩展与组织记忆证据。

4.2 自进化智能体与技能发现

近期关于自进化大语言模型智能体的研究探讨了智能体如何通过利用大语言模型分析过往任务迹来从交互经验中改进 [10, 12, 13, 14, 31, 32]。典型流程是收集先前任务的轨迹, 让大语言模型识别关键成功或失败因素, 并将其提炼为可复用的经验、规则、提示或记忆条目, 以指导未来行为。例如, ReasoningBank [12] 积累历史推理路径, 并要求大语言模型分析正确与错误的推理步骤, 将所得洞见转化为记忆条目以供后续推理使用。MemSkill [14] 研究记忆构建技能: 它调整智能体提取记忆条目的方式, 并利用下游记忆问答迹中的任务信号改进这些技能。相比之下, ERSkill 专注于构建记忆检索而非记忆构建的技能。它将查询时的记忆访问视为一种可进化的检索行为, 并引入经验树与双前沿机制以实现有效且稳定的自进化。

5 结论

本文提出了 **ERSkill**，一个以检索为核心、面向自进化且由技能引导的智能体记忆检索框架。**ERSkill** 将记忆访问表示为由共享原语库组合而成的可执行检索技能，并利用训练好的路由器选择与每个查询的信息需求相匹配的技能。为构建有效且可部署的技能，**ERSkill** 通过经验字典树和双前沿机制协同进化技能集与路由器，将能力扩展与面向路由器的部署分离开来。在三个智能体记忆基准上的实验表明，**ERSkill** 在不同骨干大语言模型上均优于强大的非进化和自进化基线，进一步的分析则展示了跨数据集迁移能力、良好的成本-性能权衡以及稳定的进化过程。这些结果凸显了检索侧进化作为长期大语言模型智能体一个有前景的方向。

参考文献

- [1] OpenAI. Gpt-4 technical report, 2024.
- [2] Gemini Team. Gemini: A family of highly capable multimodal models, 2025.
- [3] DeepSeek-AI. Deepseek-v3 technical report, 2025.
- [4] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [5] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, et al. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*, 2025.
- [6] Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, et al. Rethinking memory mechanisms of foundation agents in the second half. *arXiv preprint arXiv:2602.06052*, 2026.
- [7] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [8] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory os of ai agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25972–25981, 2025.
- [9] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, et al. Lightmem: Lightweight and efficient memory-augmented generation. *arXiv preprint arXiv:2510.18866*, 2025.
- [10] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024.
- [11] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106, 2026.
- [12] Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025.
- [13] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [14] Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026.

- [15] Anthropic. Agent skills overview. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>, 2025. Accessed: 2026-05-01.
- [16] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pages 6769–6781, 2020.
- [17] Stephen Robertson and Hugo Zaragoza. *The probabilistic relevance framework: BM25 and beyond*, volume 4. Now Publishers Inc, 2009.
- [18] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting in retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5303–5315, 2023.
- [19] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024.
- [20] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-memeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [21] Yiming Du, Hongru Wang, Zhengyi Zhao, Bin Liang, Baojun Wang, Wanjun Zhong, Zezhong Wang, and Kam-Fai Wong. Perltqa: A personal long-term memory dataset for memory classification, retrieval, and fusion in question answering. In *Proceedings of the 10th SIGHAN Workshop on Chinese Language Processing (SIGHAN-10)*, pages 152–164, 2024.
- [22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [23] OpenAI. Introducing GPT-5.4 mini and nano. <https://openai.com/index/introducing-gpt-5-4-mini-and-nano>, March 2026. Accessed: 2026-05-01.
- [24] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.
- [25] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.
- [26] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [27] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: towards llms as operating systems. 2023.
- [28] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z Pan, et al. Memory-rl: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.
- [29] Yu Wang, Ryuichi Takanobu, Zhiqi Liang, Yuzhen Mao, Yuanzhe Hu, Julian McAuley, and Xiaojian Wu. Mem- $\{\alpha\}$: Learning memory construction via reinforcement learning. *arXiv preprint arXiv:2509.25911*, 2025.
- [30] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiyang Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, et al. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. *arXiv preprint arXiv:2507.02259*, 2025.

- [31] Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.
- [32] Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R Fung. Xskill: Continual learning from experience and skills in multimodal agents. *arXiv preprint arXiv:2603.12056*, 2026.
- [33] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength Natural Language Processing in Python. 2020.
- [34] Jena D Hwang, Chandra Bhagavatula, Ronan Le Bras, Jeff Da, Keisuke Sakaguchi, Antoine Bosselut, and Yejin Choi. (comet-) atomic 2020: On symbolic and neural commonsense knowledge graphs. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 6384–6392, 2021.

目录

1 引言 1

2 方法论 3

2.1 Overview	3
2.2 Memory Storage	3
2.3 Inference	4
2.4 Skill-Router Co-Evolution	4

3 实验 6

3.1 Experimental Setup	6
3.2 Comparison Experiments	7
3.3 Ablation Study	8
3.4 Hyperparameter Study	8
3.5 Case Study of Evolution	8
3.6 Evolution Stability Analysis	9

4 相关工作 9

4.1 智能体记忆	9
4.2 自进化智能体与技能发现	9

5 结论 10

A 记忆存储细节 15

A.1 Structured Memory Construction	15
--	----

B Details for Evolution 17

B.1 Algorithm of Evolution	17
B.2 Skill Candidate Generation	18

C 证明 23

D 更多实现细节 24

D.1 Benchmarks	24
D.2 More Details	24
D.3 Rollout Reuse and Caching	25
D.4 Prompt Template	26
D.4.1 LLM-based Relation Extraction	26
D.4.2 LLM-as-a-Judge Prompt	26
D.4.3 LLM-based Skill Router	28

E 示例 29

E.1 经验字典树示例	29
-------------------	----

F 局限性与未来工作 29

A 记忆存储细节

A.1 结构化记忆构建

我们描述 ERSkill 所使用的结构化记忆的构建过程。给定交互历史 D ，ERSkill 将其编译为检索优先的记忆存储：

$$M(D) = (\mathcal{A}, \mathcal{I}, \mathcal{G}), \quad (2)$$

其中 $\mathcal{A}$ 表示一组原子级记忆记录， $\mathcal{I}$ 表示为搜索原语提供入口点的索引集合， $\mathcal{G}$ 表示扩展原语所使用的图集合。构建流程总结为：采样 $\rightarrow$ 原子 $\rightarrow$ 原子/实体嵌入 $\rightarrow$ 相似度边 $\rightarrow$ 关系边 $\rightarrow$ 运行时索引。

(3)

记忆原子。我们首先将每个原始基准样本转换为文本原子序列。分割策略取决于数据集。对于 LoCoMo，我们读取对话中的会话字段以及相应的会话时间戳，并使用轮次对作为原子粒度。LongMemEval 以类似方式处理，使用其历史会话、会话日期和会话标识符。对于 PerLTQA，我们首先将异构记忆源（包括个人资料、关系、事件和对话）扁平化为记忆记录。对于个人资料、关系和事件，每条记录直接视为一个文本原子。对于对话，每个会话视为一个文本原子。

每个文本原子随后被包装为记忆原子。我们将每个记忆原子表示为

$$a_i = (\text{atom_id}, \text{text}, \text{timestamp}, \text{entity_set}). \quad (4)$$

我们进一步为每个原子丰富若干访问信号。首先，我们使用 spaCy 包 [33] 实现的命名实体识别从原子文本中提取实体集。其次，对于 LoCoMo 和 LongMemEval，我们保留相应时间戳作为时间元数据。对于 PerLTQA，当没有可比的会话级时间戳时，时间戳字段留空。

嵌入索引。在构建原子之后，我们对原子和实体进行嵌入。对于原子嵌入，我们收集所有原子文本，并使用上下文侧检索编码器对其进行编码：

$$\mathbf{h}_i = \text{Embed}_{\text{ctx}}(a_i.\text{text}). \quad (5)$$

得到的向量作为原子嵌入存储，并用于稠密检索。我们还从原子实体集中收集所有唯一的实体字符串：

$$\mathcal{E} = \bigcup_{a_i \in \mathcal{A}} a_i.\text{entity_set}, \quad (6)$$

使用相同的嵌入接口对它们进行编码，并存储得到的实体文本和实体嵌入。这些实体嵌入支持以实体为中心的搜索和图激活。

相似度图。我们利用原子嵌入在记忆原子上构建相似度图。对于每个原子，我们计算其与所有其他原子的余弦相似度，移除自环边，保留前 k ($k = 5$ 个（在我们的实现中）最相似的原子，并丢弃非正相似度。每个保留的邻居产生一条有向边：

$$e_{i,j}^{\text{sim}} = (a_i, a_j, \text{similarity}, s_{i,j}, \text{embedding_topk}), \quad (7)$$

其中 $s_{i,j}$ 是余弦相似度得分：

$$s_{i,j} = \frac{\mathbf{h}_i^\top \mathbf{h}_j}{\|\mathbf{h}_i\|_2 \|\mathbf{h}_j\|_2}. \quad (8)$$

该图由 similarity_expand 使用，并为关系抽取提供候选邻居。

关系图。我们进一步在原子之间构建类型化的关系边。关系图包含基于规则的时间邻居边和由大语言模型提取的语义关系边。对于时间邻居边，我们按会话对原子进行分组，按轮次顺序排序，并在时间键重叠时连接相邻原子：

$$a_i \leftrightarrow a_j \quad \text{如果 } a_i.\text{时间键} \cap a_j.\text{时间键} \neq \emptyset \text{ 且 } a_i, a_j \text{ 在同一会话中相邻} . \quad (9)$$

这些边以双向方式添加，关系类型为时间邻居。该规则适用于基于会话的数据集，如 LoCoMo 和 LongMemEval。

对于语义关系边，我们从每个源原子的相似度邻居开始，保留排名靠前的候选。对于非 PerLTQA 数据集，我们仅保留时间上晚于源原子的候选邻居，以便提取的关系遵循交互历史的前向时间方向（PerLTQA 为双向）。然后将源原子及其候选邻居提供给大语言模型关系提取器。提取器被限制输出以下标签之一： $\mathcal{R} = \{ \text{改变, 原因, 理由, 受阻于, 反应, 想要, 无} \}$ 。

(10)

关系集合的灵感来源于 [34]。如果预测标签不是“无”，我们添加一条类型化的关系边：

$$e_{i,j}^{\text{rel}} = (a_i, a_j, r_{i,j}, \max(0.5, s_{i,j})), \quad (11)$$

其中 $r_{i,j} \in \mathcal{R} \setminus \{ \text{无} \}$ 是预测的关系标签， $s_{i,j}$ 是候选邻居的相似度分数。提取提示见附录 D.4.1。

运行时索引。最后，我们通过构建运行时索引来最终确定记忆存储，以实现高效的原语执行。最终确定的索引包括：原子标识到原子的映射、用于 BM25 风格词法检索的词汇与逆文档频率统计、实体到原子的倒排索引、原子到实体的映射、时间键索引、关系邻接表以及相似度邻接表。这些索引使记忆能够直接被检索原语执行。生成的记忆存储被序列化并缓存，因此后续技能执行可以复用相同的结构化记忆，而无需重新构建嵌入、图或索引。

原语接口。ERSkill 通过固定的原语库暴露结构化记忆。每个原语实现为一个 Python 函数调用，并由共享的原语执行器执行。给定当前查询、结构化记忆和当前检索状态，原语返回一组候选原子以及执行信号，执行器根据技能程序将这些候选合并或替换到活跃证据状态中。

原语库包含三类操作符。搜索原语通过 $\mathcal{I}$ 中的索引进入全局记忆。dense_search 对查询进行嵌入，并根据缓存的原子嵌入计算余弦相似度来检索原子。lexical_search 利用记忆最终确定阶段构建的词汇与逆文档频率统计执行 BM25 风格的表面形式匹配。entity_search 首先从查询中提取实体，将其与缓存的实体嵌入进行匹配，激活实体-原子索引，并通过结合实体种子分数与查询-原子语义先验的个性化 PageRank 过程对候选原子进行排序。

扩展原语通过 $\mathcal{G}$ 中的图从活跃候选原子扩展当前证据状态。similarity_expand 从当前活跃原子出发，沿预计算的相似度边扩展，并添加语义相邻的原子。relation_expand 从当前活跃原子出发，沿类型化关系边扩展，可选地受偏好关系（如原因、理由、改变或时间邻居）约束。temporal_focus_expand 检索时间戳落在显式时间范围内的原子，并可选地利用查询-原子嵌入相似度对其进行重排序；因此，它要求技能程序或上游处理步骤提供有效的 time_range。

此外，llm_process 充当处理原语而非检索原语。它读取问题、当前查询和当前证据上下文，然后将结构化变量写回检索状态，例如 preferred_relations, time_range, 一个重写的 current_query, 或一个 view_summary。这些变量随后可以控制扩展原语，例如为 relation_expand 选择关系标签，或为 temporal_focus_expand 推导时间窗口。我们在表 3 中总结了这些原语。所有原语都通过函数调用提供给大语言模型。

类型	原语	功能	搜索	entity_search	通过查询中提及或隐含的实体进行锚点检索。	搜索	lexical_search	检索具有表面形式词汇匹配的原子。	搜索	dense_search	通过查询的语义相似性检索原子。	扩展	temporal_focus_expand	围绕时间相关的原子或相邻轮次进行扩展。	扩展	similarity_expand	扩展到通过语义相似性连接的原子。	扩展	relation_expand	沿原子之间的类型化关系进行扩展。	处理	tlm_process	重写查询、过滤证据或规范化检索状态等。
----	----	----	----	---------------	----------------------	----	----------------	------------------	----	--------------	-----------------	----	-----------------------	---------------------	----	-------------------	------------------	----	-----------------	------------------	----	-------------	---------------------

表 3: ERSkill 中的原语库。搜索原语定位候选原子，扩展原语扩展证据，处理原语用于中间处理。

B 进化细节

B.1 进化算法

我们在算法 1 中总结了技能路由器协同进化的过程。

Algorithm 1 Skill-Router Co-Evolution in ERSkill

Require: Training batches $\{Q_t\}_{t=0}^{T-1}$, validation set Q_{val} , seed skill set $\mathcal{K}_{\text{seed}}$, primitive library $\mathcal{P}$, initial router parameters θ_0 , routed-gain margin γ_{route} , compactness tolerance ξ_{drop}

Ensure: Final deploy frontier $\mathcal{B}_T$, router parameters θ_T , and experience trie $\mathcal{T}_T$

- 1: Initialize $\mathcal{C}_0 \leftarrow \mathcal{K}_{\text{seed}}$, $\mathcal{B}_0 \leftarrow \mathcal{K}_{\text{seed}}$, and initialize $\mathcal{T}_0$ with the paths in $\mathcal{K}_{\text{seed}}$
- 2: Initialize the router training window $\mathcal{W}_0 \leftarrow \emptyset$
- 3: **for** $t = 0, \dots, T - 1$ **do**
- 4: Roll out each skill $\kappa \in \mathcal{C}_t$ on each query $q \in Q_t$ to obtain query-skill scores, execution traces, and ability overlaps
- 5: Write rollout results of $\mathcal{C}_t$ into $\mathcal{T}_t$
- 6: Generate candidate skills $\mathcal{U}_t$ by editing paths from $\mathcal{C}_t$ and filtering out paths already recorded in $\mathcal{T}_t$
- 7: Roll out each candidate $\kappa \in \mathcal{U}_t$ on Q_t and write its path and train-batch rollout results into $\mathcal{T}_t$
- 8: Compute the train-side temporary frontier $\tilde{\mathcal{C}}_{t+1} \leftarrow \Phi(\mathcal{C}_t \cup \mathcal{U}_t; Q_t)$
- 9: Let $\mathcal{V}_t \leftarrow \mathcal{U}_t \cap \tilde{\mathcal{C}}_{t+1}$ be the candidates retained by train-side recomputation
- 10: Roll out each retained candidate $\kappa \in \mathcal{V}_t$ on Q_{val}
- 11: Update the capability frontier $\mathcal{C}_{t+1} \leftarrow \Phi(\mathcal{C}_t \cup \mathcal{V}_t; Q_{\text{val}})$
- 12: Update $\mathcal{W}_{t+1}$ with rollout instances $(q, \mathcal{K}_q, \{r(q, \kappa)\}_{\kappa \in \mathcal{K}_q})$ collected in the current step
- 13: Continually update the router from θ_t to θ_{t+1} using mini-batches from $\mathcal{W}_{t+1}$
- 14: **if** $\mathcal{C}_{t+1} \neq \mathcal{C}_t$ **then**
- 15: Derive the deploy-update candidate set $\mathcal{H}_t \leftarrow \mathcal{U}_t \cap \mathcal{C}_{t+1}$
- 16: Construct the candidate deploy frontier $\mathcal{B}'_t \leftarrow \Phi(\mathcal{B}_t \cup \mathcal{H}_t; Q_{\text{val}})$
- 17: Compute $\Delta_{\text{route}}(\mathcal{B}'_t; \theta_{t+1}) \leftarrow \text{Routed}(\mathcal{B}'_t, \theta_{t+1}; Q_{\text{val}}) - \text{Routed}(\mathcal{B}_t, \theta_{t+1}; Q_{\text{val}})$
- 18: **if** $\Delta_{\text{route}}(\mathcal{B}'_t; \theta_{t+1}) \geq \gamma_{\text{route}}$ **or** $(\Delta_{\text{route}}(\mathcal{B}'_t; \theta_{t+1}) \geq -\xi_{\text{drop}} \text{ and } |\mathcal{B}'_t| \leq |\mathcal{B}_t|)$ **then**
- 19: Accept the deploy update and set $\mathcal{B}_{t+1} \leftarrow \mathcal{B}'_t$
- 20: **else**
- 21: Reject the deploy update and set $\mathcal{B}_{t+1} \leftarrow \mathcal{B}_t$
- 22: **end if**
- 23: **else**
- 24: Set $\mathcal{B}_{t+1} \leftarrow \mathcal{B}_t$
- 25: **end if**
- 26: Write frontier status and deploy accept/reject outcomes into $\mathcal{T}_t$
- 27: Set $\mathcal{T}_{t+1} \leftarrow \mathcal{T}_t$
- 28: **end for**
- 29: **return** $\mathcal{B}_T, \theta_T, \mathcal{T}_T$

B.2 技能候选生成

我们将技能演化过程组织为三阶段智能体工作流。首先，从训练批次中选取部分轨迹用于技能候选生成。对于每条选中的轨迹（rollout），调用分析器进行基于轨迹的案例分析。失败轨迹与成功轨迹分别由两个不同的分析器提示处理：失败轨迹分析器诊断当前技能在何处失效，包括根本原因、失败模式以及导致不匹配的关键节点；成功轨迹分析器解释技能为何与任务匹配，并识别对成功执行有实质性贡献的节点。接着，设计器将这些案例级分析结果与当前技能目录、前沿技能、原语目录、聚合技能度量、预言机性能以及近期演化历史进行汇总。基于这一综合诊断，设计器产生高层演化决策，要么保持技能集不变，要么提出新的路径候选。最后，生成器接收每个获批的设计器提案，并将其具体化为具体的技能字段，包括技能名称、描述、信息偏好以及可执行程序 JSON，同时确保生成的技能不超出提案范围，并且相对于现有规范程序具有新颖性。

Analyzer Prompt for Failed Trace

You are the analyzer for retrieval skill evolution.

角色：针对单次失败运行的逐案例分析器。仅生成基于轨迹的诊断；不得提出技能层面的行动建议。

任务：追踪证据和状态在程序执行过程中的变化，判断当前技能与任务的匹配程度，判断规范程序是否支持所声明的技能，并指定唯一一个诊断根本原因。

Rules:

- Use full trace, retrieval_metrics, evidence_view, and final_state.
- Root causes:
 - skill_capability_gap: 当前描述 / information_preference / 程序不足以完成任务，包括所需证据未被检索到的情况。
 - skill_over_broad_boundary: case appears attracted by the wrong skill; main issue is ownership or boundary clarity.
 - answer_generation_mismatch: useful evidence is present but the final answer
- 同时将失败模式分类为以下唯一一种：
 - retrieval_gap: 所需证据未被检索到或检索路径不充分。
 - synthesis_gap: 有用证据已存在，但提取、推理或答案生成仍然失败。
 - 关注当前技能与任务匹配在何处得到支持以及在何处失效。不要建议技能变更。
 - 使用 key_nodes 识别导致失败或与预期行为不匹配的节点。
 - 使用 overall_trace_judgment 总结技能的整体失败模式。
 - 使用 node_findings 解释每个节点的作用、证据或状态变化，以及观察到的不匹配、支持缺失或漂移。

Output Schema:

Return strict JSON with keys:

- case_index
- case_query
- case_groundtruth
- case_prediction
- case_chosen_skill
- root_cause
- failure_mode
- overall_trace_judgment
- program_support_status

```
- key_nodes
- node_findings
key_nodes must be a list of objects with keys:
- node_id
- primitive
- undesired_behavior
Each node_findings item must contain:
- node_id
- primitive
- role_in_program
- evidence_delta
- query_or_state_delta
- observed_problem
```

Case Input:
<case_payload>

Analyzer Prompt for Successful Trace

You are the analyzer for retrieval skill evolution.

角色：针对单个成功案例的逐案分析器。仅生成结构化成功解释；不得虚构离散的成功分类法。

任务：解释当前技能为何在该案例中与任务匹配、哪些节点对成功有实质性贡献，以及规范程序是否支持所声明的技能。

Rules:

- Use full trace, retrieval_metrics, evidence_view, and final_state.
- 保持分析叙述性和结构化；不要创建离散的成功模式标签。
- Use key_nodes to identify the nodes that materially contributed to success.
- Use overall_trace_judgment to summarize the skill's overall success pattern.
- 聚焦于技能与任务匹配在此处有效的原因；不要建议技能变更。

Output Schema:

Return strict JSON with keys:

- case_index
- case_query
- case_groundtruth
- case_prediction
- case_chosen_skill
- overall_trace_judgment
- program_support_status
- key_nodes
- node_findings

key_nodes must be a list of objects with keys:

- node_id
- primitive
- contribution

Each node_findings item must contain:

- node_id
- primitive
- role_in_program
- evidence_delta
- query_or_state_delta

Case Input:
<case_payload>

Designer Prompt

You are the designer for retrieval skill evolution.

角色：将基于轨迹的案例整合到技能集层面的诊断中，并输出最多 `<shadow_candidate_top_k>` 条高层可执行提案。

任务：以训练批次轨迹经验作为主要诊断信号，以能力层面的聚合信号作为全局约束。将它们与当前技能目录结合，形成关于覆盖缺口和可复用成功模式的综合结论，然后输出高层 `no_change` / `new_path_candidate` 提案。

分析视图：- 失败视图：`failed_trace_analyses` 是案例级别的，且已包含每个案例的 `root_cause`。对于每个失败案例，分析为何所有技能在该查询上均失败。- 成功视图：从技能级别视图读取 `success_trace_analyses`。总结每个技能的成功模式是什么样，它们锚定什么边界，以及哪些节点或证据行为使该技能有效。- 聚合能力视图：使用 `skill_metrics` 和 `oracle_acc` 来理解平均技能强度、每个技能的独特所有权，以及批量预言上限。- `root_cause` 仅作为诊断信号。不要让任何单个 `root_cause` 标签直接决定提案行动。- `failure_mode` 是主要的执行路径信号。将 `retrieval_gap` 视为检索路径或图支持可能不足的证据；将 `synthesis_gap` 视为有用证据可能已存在，而答案抽取或处理可能是主要问题的证据。- 提案必须来自对失败案例、成功模式、技能定义、规范程序、聚合能力信号和近期历史的最终综合结论。- 不要局限于一个坏样本。在提出更改之前，明确将局部失败与聚合能力信号进行协调。- 当综合结论不支持新路径时，使用 `no_change`。当当前库存仍存在真实能力缺口，且新的固定检索路径合理时，使用 `new_path_candidate`。- 当综合证据表明由这些模式构建的新路径能更好地覆盖缺失能力时，重新组合多个现有技能中有意义的模式是可接受的，且通常有益。- 对于 `retrieval_gap`，当聚合证据支持检索路径重新设计时，可以考虑搜索/扩展/图重写/原语交换更改。对于 `synthesis_gap`，当证据已存在时，可以考虑 `llm_process` 或更轻量的答案抽取更改。- 不要将搜索或扩展硬编码为默认行动。仅当综合证据支持时，才选择检索路径更改。- 在决定首先借用哪些现有模式时，积极使用聚合信号。尽可能优先借用当前技能集中相对价值较低的技能，避免干扰相对价值较高的模式，除非综合证据强烈证明有必要。- 将 `avg_acc` 和 `unique_win_ratio` 视为当前技能集内的比较信号，而非绝对阈值。- 设计者提案必须保持高层次。不要指定确切的节点 ID、图布局、序列计划或连接细节。- 每个有效的 `new_path_candidate` 都代表一个真正新的固定路径技能，而不是仅元数据或无程序更改的编辑。- 禁止不同名称下的相同程序。- 每个 `new_path_candidate` 必须是对恰好一个当前能力前沿技能的修改。- `source_skill_name` 必须仅从当前能力前沿中选择。

- Retired skills may appear in trie history for reference, but they cannot be chosen as source_skill_name.
- 优先考虑那些积极覆盖当前未覆盖查询或创造独特能力区域的修改。
- Do not propose a mild patch whose only meaningful change is adding one llm_process node.

Output Schema:

Return strict JSON with keys:

- skill_set_diagnosis: object with keys coverage_gaps, reusable_patterns, candidate_new_skills
- 提案：对象列表，包含键 action、root_cause、problem_pattern、trigger_source、intended_program_novelty、rationale、expected_gain、reasoning，并且仅对 new_path_candidate 还包含 new_skill_name 和 source_skill_name 字段约束：
 - action must be exactly one of: no_change, new_path_candidate
 - trigger_source must be exactly: oracle_hard
 - If action is new_path_candidate, new_skill_name must be a lowercase slug
 - If action is new_path_candidate, source_skill_name must be a lowercase slug from the current capability frontier
 - If action is no_change, do not provide new_skill_name
 - Use short enum-like values for action and trigger_source, not sentences, booleans, or explanations

Inputs:

Primitive catalog:

<primitive_catalog>

Existing skills with canonical programs:

<skill_catalog>

Current capability frontier parent skills:

<frontier_skill_catalog>

Trie history summary:

<trie_history_summary>

Recent evolution history:

<recent_history>

Aggregate skill metrics:

<skill_metrics>

Batch oracle capability:

<oracle_acc>

Failed trace analyses:

<failed_trace_analyses>

Success trace analyses:

<success_trace_analyses>

Generator Prompt

You are the generator for retrieval skill evolution.

角色：将一个已批准的提案具体化为结构化的技能字段。不要输出 markdown，也不要重新评判该提案。

Task:

将选定的提案视为最终决定。生成名称、描述、information_preference, 和 program_json, 使其保持在
该提案的范围内, 从 source_skill_name 作为父技能派生, 并确保生成的程序相对于现有规范程序具有
新颖性。

程序规则: - 仅使用有效的现有原语; 不要发明不支持的参数。- 不要重新解释提案。不要更改 action、
所有权或这是新路径候选的事实。- 保持 action 和 new_skill_name 与选定的提案一致。-
source_skill_name 是父技能。将其规范程序视为要修改的基础程序。- 不要将技能描述扩展到提案之
外。- 确保规范程序支持所声称的技能行为。- 在需要时优先进行真正的图编辑: 前置一个节点、在任
意位置插入一个节点、移除一个节点、交换一个原语或重写控制流。- 如果真正的问题是检索图设计,
不要默认只更改 llm_process 提示。- 当使用扩展风格的原语时, 通常在上游放置一个 llm_process 节点
以在扩展前确定控制输入。例如: 在 relation_expand, 之前选择关系标签, 在 temporal_focus_expand, 之
前推导 time_range, 在 similarity_expand, 之前确定应扩展哪个记忆或证据子集。-
temporal_focus_expand 需要显式的 time_range 输入, 并且通常本身不是第一跳原语; 在调用它之前,
优先使用上游检索步骤加上 llm_process 来推导 time_range。- 每个有效输出必须包含一个真正新的固定
路径程序, 而不是仅元数据或无程序更改的重写。- 不要产生唯一有意义的结构性更改是添加一个
llm_process 节点的提案。

少样本示例: 示例 1 选定的提案: { "action": "new_path_candidate", "new_skill_name":
"semantic-surface-clue", "reasoning": "当前技能检索语义相关的证据, 但经常错过包含确切答案的措辞。
保持相同的所有者, 但通过第二个检索步骤来收紧它。" } 良好的生成器行为: - 保持提案的高层意图-
将图具体化为真正的检索路径编辑, 例如 dense_search 后跟 lexical_search- 不要仅用 llm_process 提示重
写来替代更改

示例 2 选定提案: { "动作": "new_path_candidate", "new_skill_name": "实体关系链", "推理": "当前技
能找到了正确的实体, 但证据链过浅, 未沿实际连接答案的关系进行扩展。保留相同所有者并延长检
索路径。" } 良好生成器行为: - 保持技能标识 - 保留检索所有者 - 将图具体化为 entity_search 后接
relation_expand, 或 entity_search 后接 relation_expand 和 lexical_search, 当关系扩展仍需表面过滤时 -
不要仅用答案侧 llm_process 步骤替换更改

示例 3 选定提案: { "动作": "new_path_candidate", "new_skill_name": "受控时间扩展", "推理": "技能需要受控扩展, 但扩展输入无法直接从原始问题中获得。添加结构, 首先确定控制信号, 然后进行扩展。" } 良好生成器行为:

- 保持提案高层次, 并将其转化为受控图编辑

```
or time_range
- then use that control output to drive relation_expand or temporal_focus_expand
- do not stop at llm_process as the final fix when the proposal requires
controlled expansion
```

序列化规则:

- program_json must be a JSON object, not a string.
- 必须使用 json.dumps 序列化, 并使用 json.loads 解析。
- 使用转义的 JSON 安全字符串; 不要嵌入不安全的嵌套 JSON 示例。
- 尽可能使用散文示例而非字面嵌入 JSON。

Required Output:

Return strict JSON with keys:

- action
- source_skill_name
- new_skill_name
- description
- information_preference
- program_json
- reasoning

Inputs:

Selected proposal:
<proposal>

Primitive catalog:
<primitive_catalog>

Existing skills:
<skill_catalog>

Current skill markdown:
<current_markdown>

C 证明

命题 C.1 (预言机安全的两级前沿更新。命题 2.1 的重述)。将预言机覆盖率记为 $\text{OCov}(\mathcal{K}; \mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} g_{\mathcal{K}}(q)$ 。对于每个进化步骤 t , ERSkill 满足 $\text{OCov}(\mathcal{C}_{t+1}; \mathcal{Q}_{\text{val}}) \geq \text{OCov}(\mathcal{C}_t; \mathcal{Q}_{\text{val}})$ 且 $\text{OCov}(\mathcal{B}_{t+1}; \mathcal{Q}_{\text{val}}) \geq \text{OCov}(\mathcal{B}_t; \mathcal{Q}_{\text{val}})$ 。

证明。令 $r(q, \kappa) \in [0, 1]$ 表示技能 κ 在查询 q 上获得的验证分数。对于任意技能集 $\mathcal{K}$, 定义其在 $\mathcal{Q}_{\text{val}}$ 上的预言机分数概况为

$$g_{\mathcal{K}}(q) = \max_{\kappa \in \mathcal{K}} r(q, \kappa), \quad q \in \mathcal{Q}_{\text{val}}. \quad (12)$$

预言机覆盖率分数是验证查询上的平均预言机分数:

$$\text{OCov}(\mathcal{K}; \mathcal{Q}_{\text{val}}) = \frac{1}{|\mathcal{Q}_{\text{val}}|} \sum_{q \in \mathcal{Q}_{\text{val}}} g_{\mathcal{K}}(q). \quad (13)$$

我们首先证明 Φ 在用于重新计算的批次上保持预言机分数概况。考虑一个具有活动集 $\mathcal{K}_{\text{act}}$ 的剪枝步骤。仅当移除技能 κ 不会降低任何验证查询上可达到的最佳分数时，该技能才会被移除：

$$\max_{\kappa' \in \mathcal{K}_{\text{act}} \setminus \{\kappa\}} r(q, \kappa') = \max_{\kappa' \in \mathcal{K}_{\text{act}}} r(q, \kappa') \quad \text{for all } q \in \mathcal{Q}_{\text{val}}. \quad (14)$$

因此，

$$g_{\mathcal{K}_{\text{act}} \setminus \{\kappa\}}(q) = g_{\mathcal{K}_{\text{act}}}(q) \quad \text{对于所有 } q \in \mathcal{Q}_{\text{val}}. \quad (15)$$

将相同的论证应用于 Φ 中的所有剪枝步骤，可得

$$g_{\Phi(\mathcal{K}; \mathcal{Q}_{\text{val}})}(q) = g_{\mathcal{K}}(q) \quad \text{for all } q \in \mathcal{Q}_{\text{val}}. \quad (16)$$

Averaging over $\mathcal{Q}_{\text{val}}$ yields

$$\text{OCov}(\Phi(\mathcal{K}; \mathcal{Q}_{\text{val}}); \mathcal{Q}_{\text{val}}) = \text{OCov}(\mathcal{K}; \mathcal{Q}_{\text{val}}). \quad (17)$$

对于能力前沿，令 $\mathcal{K}^{\text{cap}} = \mathcal{C}_t \cup \mathcal{V}_t$ 且 $\mathcal{C}_{t+1} = \Phi(\mathcal{K}_{t+1}^{\text{cap}}; \mathcal{Q}_{\text{val}})$ 。根据方程 17，

$$\text{OCov}(\mathcal{C}_{t+1}; \mathcal{Q}_{\text{val}}) = \text{OCov}(\mathcal{C}_t \cup \mathcal{V}_t; \mathcal{Q}_{\text{val}}) \geq \text{OCov}(\mathcal{C}_t; \mathcal{Q}_{\text{val}}). \quad (18)$$

对于部署前沿，令 $\mathcal{H}_t = \mathcal{U}_t \cap \mathcal{C}_{t+1}$ 为用于更新部署前沿的候选集，并令 $\mathcal{B}'_t = \Phi(\mathcal{B}_t \cup \mathcal{H}_t; \mathcal{Q}_{\text{val}})$ 为候选部署前沿。若部署更新被接受，则 $\mathcal{B}_{t+1} = \mathcal{B}'_t$ 。根据方程 16，

$$g_{\mathcal{B}_{t+1}}(q) = g_{\mathcal{B}'_t}(q) = g_{\mathcal{B}_t \cup \mathcal{H}_t}(q) \geq g_{\mathcal{B}_t}(q) \quad \text{for all } q \in \mathcal{Q}_{\text{val}}. \quad (19)$$

对 $\mathcal{Q}_{\text{val}}$ 取平均可得

$$\text{OCov}(\mathcal{B}_{t+1}; \mathcal{Q}_{\text{val}}) \geq \text{OCov}(\mathcal{B}_t; \mathcal{Q}_{\text{val}}). \quad (20)$$

若部署更新被拒绝，ERSkill 保留先前的部署前沿，即 $\mathcal{B}_{t+1} = \mathcal{B}_t$ 。因此，

$$\text{OCov}(\mathcal{B}_{t+1}; \mathcal{Q}_{\text{val}}) = \text{OCov}(\mathcal{B}_t; \mathcal{Q}_{\text{val}}). \quad (21)$$

结合方程 20 与 21，可得

$$\text{OCov}(\mathcal{B}_{t+1}; \mathcal{Q}_{\text{val}}) \geq \text{OCov}(\mathcal{B}_t; \mathcal{Q}_{\text{val}}). \quad (22)$$

结合方程 18 与 22，证明完成。 $\square$

D 更多实现细节

D.1 基准

LoCoMo [19] LoCoMo 包含多会话对话历史，涵盖四种问题类型：多跳、时序、开放域和单跳问题。遵循先前工作 [9, 8]，我们排除了对抗性问题的。我们的划分包含 233 个训练样本、152 个验证样本和 314 个测试样本。

LongMemEval [20] LongMemEval 是一个用于评估聊天助手长期交互记忆的基准。每个实例包含带时间戳的用户-助手历史会话、用户问题、问题时间以及参考答案或评分标准。我们使用 LongMemEval-S 部分进行评估。我们的划分包含 205 个训练样本、98 个验证样本和 197 个测试样本。

PerLTQA [21] PerLTQA 是一个个人长期记忆问答数据集，旨在评估模型如何在对话中利用异构记忆源。它涵盖语义记忆（包括世界知识、个人资料和社会关系）和情景记忆（包括事件和对话）。我们的划分包含 439 个训练样本、272 个验证样本和 483 个测试样本。

D.2 更多细节

我们使用 gpt-4o-mini 作为大语言模型评判器。所有实验均在配备四块 RTX PRO6000 Blackwell GPU 的服务器上进行。路由增益边际 γ_{route} 在 LoCoMo、LongMemEval 和 PerLTQA 上分别设置为 0.00、0.00 和 0.02。紧凑性容差 ξ_{drop} 在 LoCoMo、LongMemEval 和 PerLTQA 上分别设置为 0.15、0.15 和 0.05。在附录 D.3 中，我们还描述了如何通过将 rollout 结果视为可重用记录来降低训练成本。

D.3 Rollout 重用与缓存

如果在预言机评估、路由器训练、前沿重计算和部署验证过程中反复执行相同的查询-技能对，技能演化可能会变得代价高昂。在我们的实现中，通过将展开结果视为可复用的记录来降低这一成本。其核心原则是：仅在引入新信息时才执行检索和答案生成展开；后续的训练和选择步骤尽可能在缓存的查询-技能结果上进行操作。

批次级预言机复用。对于每个训练批次 Q_t ，ERSkill 首先在当前能力前沿 C_t 上评估每个技能在每个查询上的表现。这产生一个批次级预言机表，其中每个条目存储查询-技能对的预测、执行迹、证据状态和评估分数。随后，该表被复用于识别预言机最优技能、构建路由器监督信号、记录训练迹、计算前沿统计信息以及收集成本信息。因此，一旦某个技能在批次中的某个查询上被评估过，后续组件就会读取其存储的结果，而不会重新运行该技能。

无需重新展开的路由器重放。路由器更新后，通过在现有预言机表上重放路由器决策来计算路由训练或验证性能。给定一个查询和一个候选技能集，路由器选择一个技能，ERSkill 从相应的预言机表中检索所选技能的缓存展开载荷。这避免了为路由器所选技能进行额外的展开。因此，路由性能估计仅需要轻量级的路由器推理和表查找，而无需再次执行检索和生成。

增量候选评估。当生成新的技能候选时，ERSkill 不会重新评估所有现有技能。相反，它在相关的训练或验证查询上对该候选执行单技能展开，然后将候选结果合并到现有预言机表中。前沿重计算、候选过滤和部署前沿构建随后作为合并表上的离线分数和集合操作执行。这将许多重复评估转化为对先前收集的查询-技能分数的数据复用。

验证预言机维护。ERSkill 在整个进化过程中维护一个验证预言表。初始化时，种子技能在验证集上进行评估。当新候选通过训练侧重计算并需要验证时，仅将这些新候选在验证查询上展开。其结果合并到维护的验证预言表中，同时复用之前的验证展开结果。这对于部署前沿更新尤为有用，因为此类更新需要反复比较不同候选部署集下的路由性能。

基于内容哈希的技能评估缓存。对于独立评估每个技能的评估例程，ERSkill 使用技能内容缓存。每个缓存的技能负载与技能 Markdown 内容的哈希关联。仅当当前技能内容哈希与缓存哈希匹配时，才复用缓存的展开结果；否则重新运行该技能。这避免了技能未变化时的冗余展开，同时防止技能名称相同但内容已变时的错误复用。

内存与路由器侧缓存。ERSkill 还缓存跨展开共享的计算。结构化内存存储构建一次，并在技能执行期间从缓存加载，避免重复的原子化、实体抽取、嵌入、关系抽取和图构建。对于路由器，技能 Markdown 嵌入被缓存，仅当新技能出现或现有技能内容变化时才刷新。路由器训练窗口还存储来自历史预言展开的紧凑监督记录，使后续路由器更新能够复用之前的监督，而无需重新执行底层技能。

总体而言，这些实现策略确保昂贵的展开主要针对新引入的查询-技能对执行。现有前沿技能、验证结果、路由器监督和内存表示通过预言表、增量合并、基于重放的路由评估和内容感知缓存得以复用。

D.4 提示模板

D.4.1 基于大语言模型的关系抽取

对于基于大模型的关系抽取，ERSkill 要求大模型为源原子到每个候选邻居原子分配一个类型化关系。对于 LoCoMo 和 LongMemEval，我们额外加入时间提示“请记住，[源原子] 发生在所有列出的 [邻居原子] 之前。”；对于 PerLTQA，由于记忆来源并不总是具有可比的时间顺序，因此省略此提示。

LLM-based Relation Extraction Prompt

Your task is to find the relation from [Source Atom] to each [Neighbor Atom].
<temporal_hint>

对于每个邻居，判断以下关系之一是否明确成立：

1. Changed: when events in [Source Atom] changed to events in [Neighbor Atom]
2. Cause: when events in [Source Atom] caused events in [Neighbor Atom]
3. Reason: when events in [Source Atom] are due to events in [Neighbor Atom]
4. 受阻于：当 [邻居原子] 中的事件可能受到 [源原子] 中事件的阻碍，反之亦然
5. 反应：当由于 [源原子] 中的事件，主体产生了 [邻居原子] 中所述的感觉
6. 想要：当由于 [源原子] 中的事件，主体希望 [邻居原子] 中的事件发生

如果某个邻居不属于上述任何关系，则输出“无”。只有在有明确证据符合定义时才选择关系。不要对给定原子进行过度推断。注意主体是谁。不要混淆 [源原子] 和 [邻居原子] 的角色。

Source memory:
<source_atom_text>

Neighbor memories:
<neighbor_atom_json_list>

Return JSON only in this format:
{
 "relations": [
 {
 "atom_id": "<neighbor atom id>",
 "label": "Changed|Cause|Reason|HinderedBy|React|Want|none"
 }
]
}

D.4.2 大模型作为评审提示

我们在本节中总结了每个基准所使用的大模型作为评审提示。

LoCoMo Judge Prompt

你的任务是判断一个针对问题的生成答案是否正确。你将获得以下数据：

- (1) a question (posed by one user to another user),
- (2) a 'gold' (ground truth) answer,
- (3) 一个生成答案，你需要将其评分为正确或错误。

问题的要点是询问一个用户基于他们之前的对话应该了解另一个用户的某件事。标准答案通常是一个简洁且简短的答案，包含所提及的主题，例如：问题：你还记得我上次去夏威夷时买了什么吗？标准答案：一条贝壳项链

生成答案可能会长得多，但评分时应宽松——只要它触及与标准答案相同的主题，就应视为正确。

对于与时间相关的问题，标准答案将是一个具体的日期、月份、年份等。生成答案可能会长得多或使用相对时间表述（如“上周二”或“下个月”），但评分时应宽松——只要它指向与标准答案相同的日期或时间段，就应视为正确。即使格式不同（例如，“5 月 7 日”与“7 May”），如果日期相同，也应视为正确。

```
Now it's time for the real question:  
Question: <question>  
Gold answer: <gold_answer>  
Generated answer: <generated_answer>
```

首先，用一句话简短解释你的推理，然后以“正确”或“错误”结束。不要在回答中同时包含“正确”和“错误”，否则会破坏评估脚本。

只需以 JSON 格式返回分数，键为“score”。使用 `{"score": 1}` 表示正确，`{"score": 0}` 表示错误。

LongMemEval Judge Prompt for Single-Session User / Single-Session Assistant / Multi-Session

我会给你一个问题、一个正确答案和一个模型回答。请判断该回答是否包含正确答案。如果回答与正确答案等价或包含得到正确答案的所有中间步骤，则判定为正确。如果回答仅包含答案所需信息的一部分，则判定为错误。只返回带有键“score”的有效 JSON：如果模型回答正确，返回 `{"score": 1}`；否则返回 `{"score": 0}`。

```
Question: <question>  
  
Correct Answer: <correct_answer>  
  
Model Response: <model_response>
```

LongMemEval Judge Prompt for Temporal Reasoning

我会给你一个问题、一个正确答案和一个模型的回答。请判断该回答是否包含正确答案。如果回答与正确答案等价，或包含得到正确答案的所有中间步骤，则判定为正确。如果回答仅包含答案所需信息的一部分，则判定为不正确。对于天数、周数或月数的差一错误，不要扣分。例如，答案为 18 天时预测为 19 天仍算正确。只返回有效的 JSON，键为“score”：如果模型回答正确则返回 `{"score": 1}`；否则返回 `{"score": 0}`。

```
Question: <question>  
  
Correct Answer: <correct_answer>  
  
Model Response: <model_response>
```


LongMemEval Judge Prompt for Knowledge Update

我将给你一个问题、一个正确答案和一个模型的回答。请判断该回答是否包含正确答案。如果回答包含之前的信息以及更新后的答案，只要更新后的答案是所需答案，该回答就算正确。只返回有效的 JSON，键为 "score"：如果模型回答正确，返回 {"score": 1}；否则返回 {"score": 0}。

Question: <question>

Correct Answer: <correct_answer>

Model Response: <model_response>

LongMemEval Judge Prompt for Single-Session Preference

我会给你一个问题、一个用于期望个性化回答的评分标准，以及一个模型的回答。请判断该回答是否满足期望的回答。模型不需要反映评分标准中的所有要点。只要回答正确回忆并利用了用户的个人信息，回答就是正确的。只返回有效的 JSON，键为 "score"：如果模型回答正确，返回 {"score": 1}；否则返回 {"score": 0}。

Question: <question>

Rubric: <rubric>

Model Response: <model_response>

PerLTQA Judge Prompt

我将给你一个问题关于角色个人长期记忆的问题、一个正确答案，以及一个模型的回答。请判断该模型的回答是否正确回忆并陈述了角色记忆中的关键事实。如果回答在语义上与正确答案等价，即使措辞不同，也判定为正确。如果问题要求多个关键事实而回答遗漏了重要部分，则判定为不正确。如果回答添加了不支持的或矛盾的事实，改变了答案的含义，则判定为不正确。重点关注所回忆的个人资料、关系、事件或对话记忆的事实正确性，而非风格差异。只返回有效的 JSON，键为 "score"：如果回答正确，返回 {"score": 1}；否则返回 {"score": 0}。

Question: <question>

Correct Answer: <correct_answer>

Model Response: <model_response>

D.4.3 基于大语言模型的技能路由器

LLM-based Skill Router Prompt

从提供的技能中为问题选择最佳检索技能。仅从提供的技能中选择。可用检索技能：<skill_candidates>

Question: <question>

